Session 6: Nonlinear regression

Neural Network

Instructor: Mohammad Arashi (FUM, UP)

دانشگاه فردوسی مشهد
Ferdowsi University of Mashhad

UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA

Motivating example

The human visual system is one of the wonders of the world. Consider the following sequence of handwritten digits: Most people effortlessly recognize those digits as 504192.

The difficulty of visual pattern recognition becomes apparent if you attempt to write a computer program to recognize digits like those above.

http://neuralnetworksanddeeplearning.com/chap1.html

Motivation Example 1 - Continued

Neural networks approach the problem under study nicely. The idea is to take a large number of handwritten digits, known as **training** examples and then develop a system which can learn from those training examples. In other words, the neural network uses the examples to automatically infer rules for recognizing handwritten digits.

Furthermore, by *increasing the number of training examples*, the network can learn more about handwriting, and so **improve its accuracy**. So while I've shown just 100 training digits above, perhaps we could build a better handwriting recognizer by using thousands or even millions or billions of training examples.

http://neuralnetworksanddeeplearning.com/chap1.html

Motivation

Neural Network (or Artificial Neural Network, ANN) has the ability to learn by examples. ANN is an information processing model inspired by the biological neuron system. It is composed of a large number of highly interconnected processing elements known as the neuron to solve problems. It follows the non-linear path and process information in parallel throughout the nodes.

A neural network is a complex adaptive system. Adaptive means it has the ability to change its internal structure by adjusting weights of inputs.

Motivation - continued

The neural network was designed to solve problems which are easy for humans and difficult for machines such as identifying pictures of cats and dogs, identifying numbered pictures. These problems are often referred to as **pattern recognition**. Its application ranges from optical character recognition to object detection.

Motivation

Human mind - Good at image recognition, pattern recognition etc
Computer - Good at arithmetic calculations

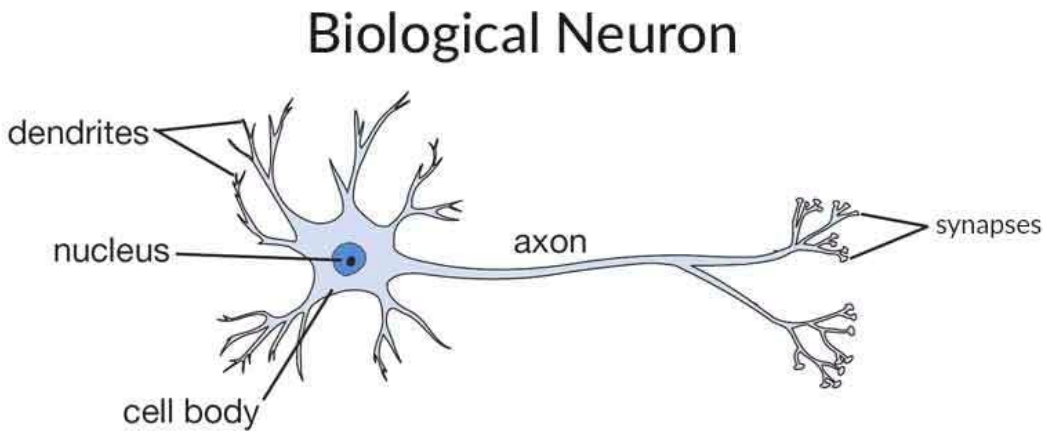
$$2574304 \times e^{354} \div \tan 5.1\pi$$

Introduction to Neural Network

NN algorithms are inspired by the human brain to performs a particular task or functions. NN perform computations through a process by learning. The neural network is a of connected input/output units in which each connection has a weight associated with it. In the learning phase, the network learns by adjusting the weights to predict the correct class label of the given inputs.

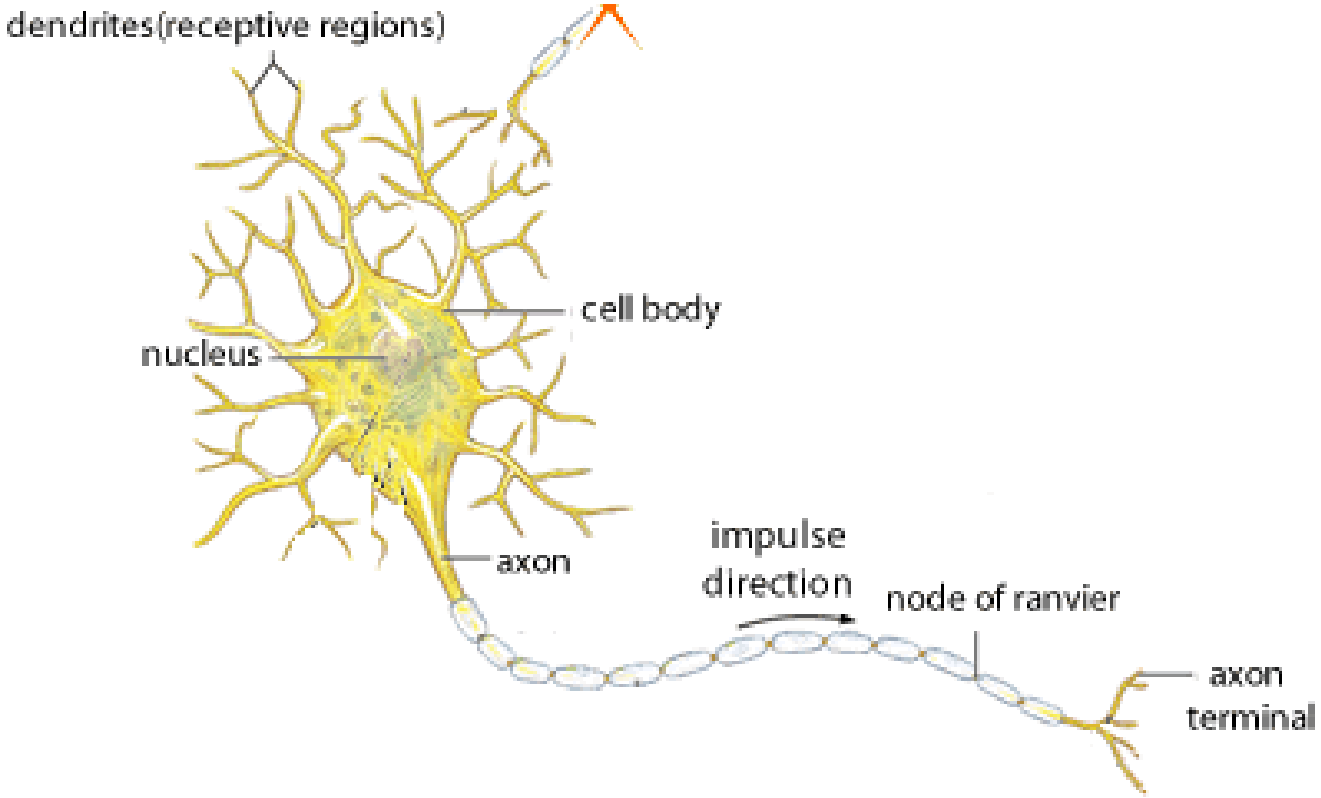
Origin of name of Neural Network

To understand neural networks, let's first explore the inspiration behind ANN - our brains.



The human brain consists of billions of neural cells that process information. Each neural cell considered a simple processing system. The Interconnected web of neurons known as biological neural network transmits information through electrical signals. This parallel interactive system makes the brain to think and process information. Dendrites of a neuron receive input signals from another neuron and respond output based on those inputs to an axon of some other neuron. Based on those inputs, fire an output signal via an axon.

Dendrites receive signals from other neurons. Cell body sums all the inputs signals to generate output. Synapses is a point of interaction neurons. It transmits electrical or chemical signals to another neuron. Synapse is derived from the Greek word which means conjunction.



The first NN from Mathematical Aspects

The McCulloch-Pitts neural model is also known as **linear threshold gate**. It is a neuron of a set of inputs $x_1, x_2, \ldots, x_m$ and one output y. The linear threshold gate simply classifies the set of inputs into two different classes. Thus the output y is binary. Such a function can be described mathematically using these equations:

$$\text{Sum} = \sum_{i=1}^{n} x_i w_i, \tag{2.1}$$

$$y = f(\text{Sum}) \tag{2.2}$$

where $w_1, w_2, \ldots, w_m$ are weight values normalized in the range of either $(0, 1)$ or $(-1, 1)$ and associated with each input line, Sum is the weighted sum, and T is a threshold constant. The function f is a linear step function at threshold T as shown in Figure 1.

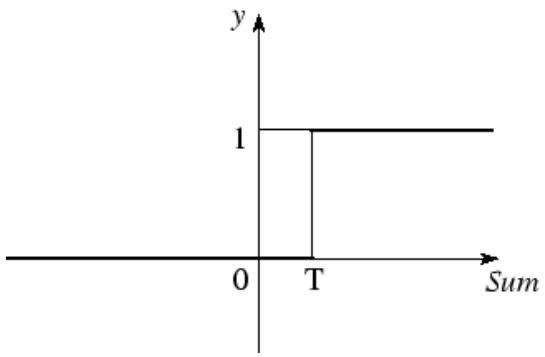


Figure: Linear Threshold Function

Logistic Regression

To give an idea how neural networks works, we briefly review logistic regression.

Goal: If we have $\boldsymbol{x}$, what is $\hat{y} = P(Y = 1|\boldsymbol{x})$ $(0 \leq \hat{y} \leq 1)$.

Components we have in our hand: input: $\boldsymbol{x}$, output: y

Parameters that should be found: $\boldsymbol{w}$ and $b \in \mathbb{R}$

approaches:

(1) Consider a kind of linear function of $\boldsymbol{x}$ as:

$$\hat{y} = \boldsymbol{x}\boldsymbol{w} + b$$

This isn't a very good algorithm for binary classification because you want $\hat{y}$ to be the chance that $y = 1$. So, y should really be between 0 and 1, and it is difficult to enforce that because $\boldsymbol{x}\boldsymbol{w} + b$ can be much bigger than one or it can even be negative, which doesn't make sense for probability.

(2) In logistic regression, our output is instead going to be

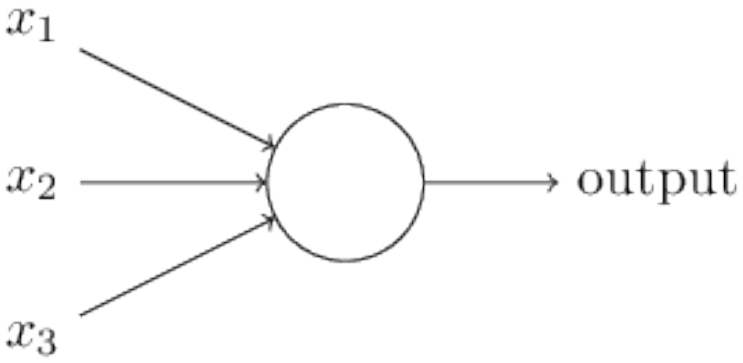
$$\hat{y} = f(\boldsymbol{x}\boldsymbol{w} + b)$$

where $f(\cdot)$ is a sigmoid function with $f(x) = \frac{1}{1+\exp\{-x\}}$

Perceptrons

Using the model **sigmoid neuron** is very common. But, to understand why sigmoid neurons are defined the way they are, it's worth taking the time to first understand perceptrons.

So how do perceptrons work? A perceptron takes several binary inputs, $x_1, x_2, \ldots$, and produces a single binary output:



Rosenblatt proposed a simple rule to compute the output. He introduced weights, w_1, w_2, ..., real numbers expressing the importance of the respective inputs to the output. The neuron's output, 0 or 1, is determined by whether the weighted sum $\sum \lim_j w_j x_j$ is less than or greater than some threshold value. Just like the weights, the threshold is a real number which is a parameter of the neuron. To put it in more precise algebraic terms:

$$\text{output} = \begin{cases} 0 & \text{if} \sum \lim_j w_j x_j \leq \text{threshold} \\ 1 & \text{if} \sum \lim_j w_j x_j > \text{threshold} \end{cases} \tag{2.3}$$

That's all there is to how a perceptron works! That's the basic mathematical model.

An example

Suppose the weekend is coming up, and you have heard that there is going to be a cheese festival in your city. You like cheese, and are trying to decide whether or not to go to the festival. You might make your decision by weighing up three factors, can represent them by corresponding binary variables, also.

(1) Is the weather good? ($x_1 = 1$ if the weather is good, and $x_1 = 0$ if the weather is bad)

(2) Does your friend want to accompany you? ($x_2 = 1$ if your friend wants to go, and $x_2 = 0$ if not.)

(3) Is the festival near public transit? (You don't own a car). ($x_3 = 1$ if the festival is near public transit and $x_3 = 0$ if not)

An example cont.

Now, suppose you absolutely adore cheese, so much so that you are happy to go to the festival even if your friend is uninterested and the festival is hard to get to. But perhaps you really loathe bad weather, and there's no way you'd go to the festival if the weather is bad.
You can use **perceptrons** to model this kind of decision-making. One way to do this is to choose a weight $w_1 = 6$ for the weather, and $w_2 = 2$ and $w_3 = 2$ for the other conditions. If the answers to all questions are "yes", So we have

$$(w_1 \times x_1) + (w_2 \times x_2) + (w_3 \times x_3) = (6 \times 1) + (2 \times 1) + (2 \times 1) = 10 \quad (2.4)$$

Suppose you choose a threshold of 5 for the perceptron. Thus, based on Eq. (2.3), we decide to go to the festival because the weighted sum is grater than the threshold.

An example cont.

In this example, the larger value of w_1 indicates that the weather matters a lot to you, much more than whether your friend joins you, or the nearness of public transit.

With these choices, the perceptron implements the desired decision-making model, outputting 1 whenever the weather is good, and 0 whenever the weather is bad. It makes no difference to the output whether your friend wants to go, or whether public transit is nearby.

By varying the weights and the threshold, we can get different models of decision-making. For example, suppose we instead choose a threshold of 3. Then the perceptron would decide that you should go to the festival whenever the weather was good or when both the festival was near public transit and your friend was willing to join you. In other words, it'd be a different model of decision-making. Dropping the threshold means you're more willing to go to the festival.

An example cont.

Remember, we supposed that $w_1 = 6$, $w_2 = 2$, and $w_3 = 2$

x_1	x_2	x_3	$w_1 \times x_1$	$w_2 \times x_2$	$w_3 \times x_3$	$\sum w_i x_i$	threshold 1	y	threshold 2	y
1	1	1	$6 \times 1 = 6$	$2 \times 1 = 2$	$2 \times 1 = 2$	$6 + 2 + 2 = 10$	5	1	3	1
1	1	0	$6 \times 1 = 6$	$2 \times 1 = 2$	$2 \times 0 = 0$	$6 + 2 + 0 = 8$	5	1	3	1
1	0	1	$6 \times 1 = 6$	$2 \times 0 = 0$	$2 \times 1 = 2$	$6 + 0 + 2 = 8$	5	1	3	1
1	0	0	$6 \times 1 = 6$	$2 \times 0 = 0$	$2 \times 0 = 0$	$6 + 0 + 0 = 6$	5	1	3	1
0	1	1	$6 \times 0 = 0$	$2 \times 1 = 2$	$2 \times 1 = 2$	$0 + 2 + 2 = 4$	5	0	3	1
0	1	0	$6 \times 0 = 0$	$2 \times 1 = 2$	$2 \times 0 = 0$	$0 + 2 + 0 = 2$	5	0	3	0
0	0	1	$6 \times 1 = 6$	$2 \times 0 = 0$	$2 \times 1 = 2$	$0 + 0 + 2 = 2$	5	0	3	0
0	0	0	$6 \times 0 = 0$	$2 \times 0 = 0$	$2 \times 0 = 0$	$0 + 0 + 0 = 0$	5	0	3	0

See Lab16.

Perceptron redefined

The condition $\sum_j w_j x_j > \text{threshold}$ is cumbersome. We can make two notational changed to simplify it.

(1) Write $\sum_j w_j x_j$ as a dot product, $\mathbf{w} \cdot \boldsymbol{x} \approx \sum_j w_j x_j$, where $\boldsymbol{w}$ and $\boldsymbol{x}$ are vectors whose components are the weights and inputs, respectively.

(2) Move the threshold to the other side of the inequality, and replace it by what is known as the perceptron's **bias**, $b \approx -\text{threshold}$. Using the bias instead of the threshold, the perceptron rule can be rewritten as

$$\text{output} = \begin{cases} 0 & \text{if} \quad \boldsymbol{w} \cdot \boldsymbol{x} + \boldsymbol{b} \leq 0 \\ 1 & \text{if} \quad \boldsymbol{w} \cdot \boldsymbol{x} + \boldsymbol{b} > 0 \end{cases}$$

You can think of the bias as a measure of how easy it is to get the perceptron to output a 1. Or to put it in more biological terms, the bias is a measure of how easy it is to get the perceptron to **fire**.

For a perceptron with a really big bias, it's extremely easy for the perceptron to output a 1. But if the bias is very negative, then it's difficult for the perceptron to output a 1.

Single layer perceptron (SLP)

The simplest type of perceptron has a single layer of weights connecting the inputs and output. In this way, it can be considered the simplest kind of **feed-forward network**. In a feed forward network, the information always moves in one direction; it never goes backwards.

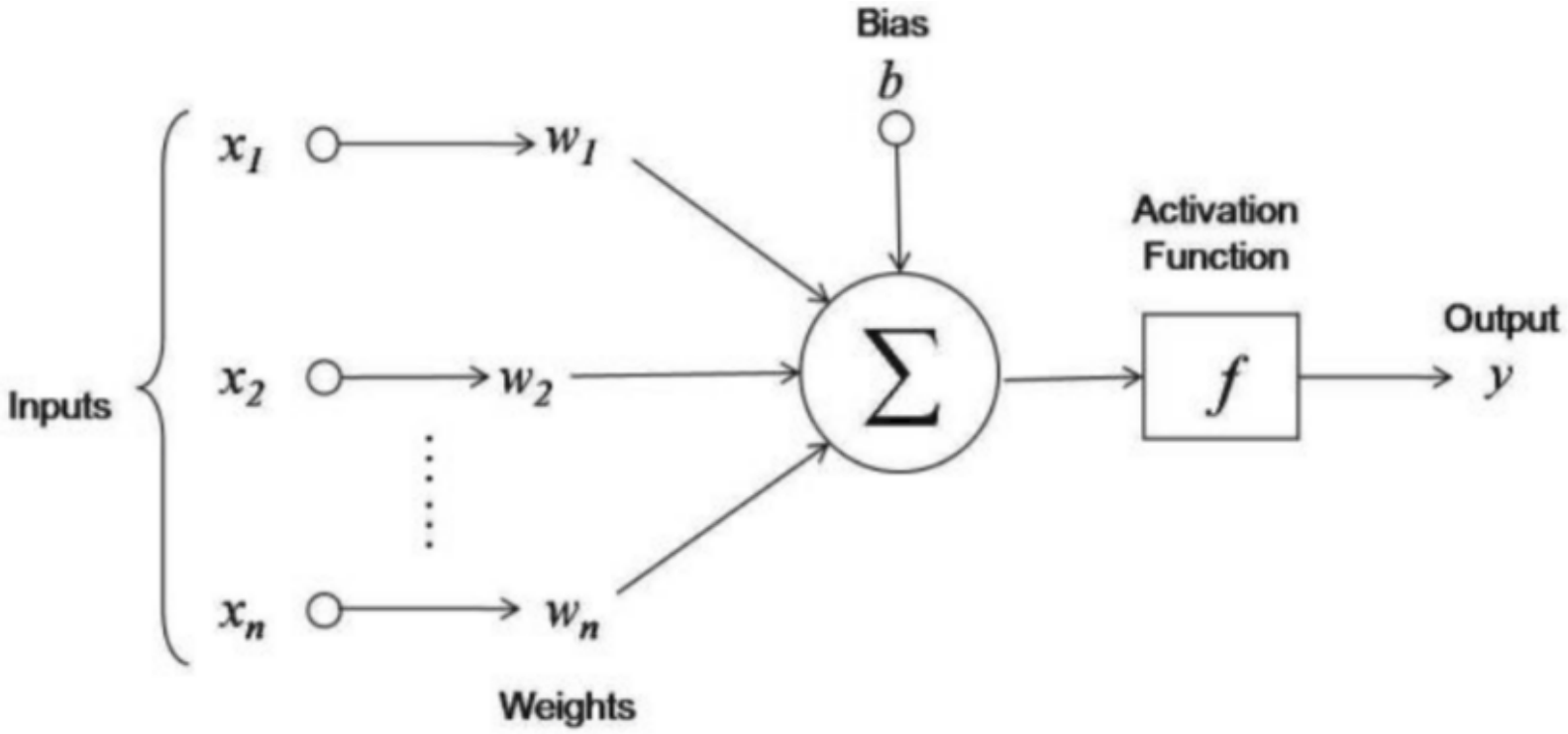


Figure: The single layer perceptron

SLP cont.

Figure shows a single-layer perceptron for easier conceptual grounding and clarification into multi-layer perceptron. Single layer perceptron represents the n weights that are seen as a set of synapses or connecting links between one layer and another layer within the network. This parameter indicates how important each feature (x_i) is. Below is the adder function of features of the input multiplied by their respective synaptic connection $u = \sum_{i=1}^{n} w_i x_i$.

The bias, b, acts as an affine transformation to the output of the adder function u giving v , the induced local field as $v = u + b$

The activation function defines the output of a neuron in terms of the induced local field v as $y = f(u + b)$

In general, it can be written as

$$\text{Output} = f(\sum(\text{Weights} \times \text{Inputs}) + \text{Bias}) \quad \text{or} \quad y = f(\sum_{i=1}^{n}(w_i x_i) + b)$$

$x_1, x_2, \ldots, x_n$ are input variables
$w_1, w_2, \ldots, w_n$ are weights of respective inputs
b is the bias, which is summed with the weighted inputs to form the net inputs.
Bias and weights are both adjustable parameters of the neuron.
Parameters are adjusted using some learning rules.
y is the output of a neuron can range from $-\infty$ to $+\infty$, **in general**. The neuron does not know the boundary. So we need a mapping mechanism between the input and output of the neuron.
This mechanism of mapping inputs to output is known as **Activation Function**. So,
f is an activation function.

Activation function

Activation function defines the output of a neuron in terms of a local induced field. Here are various types of activation functions:

Threshold (or Step or Heaviside) function: This Figure indicates that either the neuron is fully active or not.

$$f(v) = \begin{cases} 0 & \text{if} \quad v \leq \mathrm{T} \\ 1 & \text{if} \quad v > \mathrm{T} \end{cases}$$

Why is it used?

Some cases call for a function which applies a hard thresold: either the output is precisely a single value, or not. The other functions weve looked at have an intrinsic probabilistic output to them i.e. a higher output in decimal format implying a greater probability of being 1 (or a high output). The step function does away with this opting for a definite high or low output depending on some threshold on the input T.

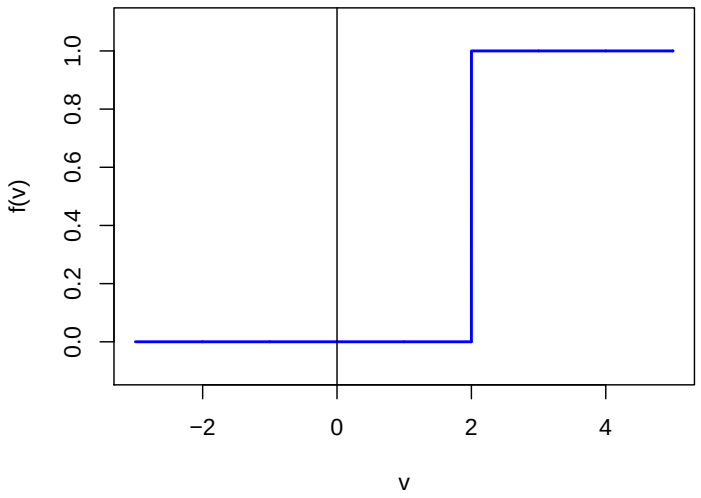


Figure: The threshold function for $T = 2$

Activation function cont.

Sigmoid (or Fermi or Logistic) Function: It is nonlinear in nature that helps to increase the performance making sure that small changes in the weights and bias causes small changes in the output of the neuron.

$$f(v) = \frac{1}{1 + \exp\{v\}}$$

Why is it used?

This function maps the input to a value between 0 and 1 (but not equal to 0 or 1). This means the output from the node will be a high signal (if the input is positive) or a low one (if the input is negative).

The sigmoids natural threshold is 0.5, meaning that any input that maps to a value above 0.5 will be considered high (or 1) in binary terms.

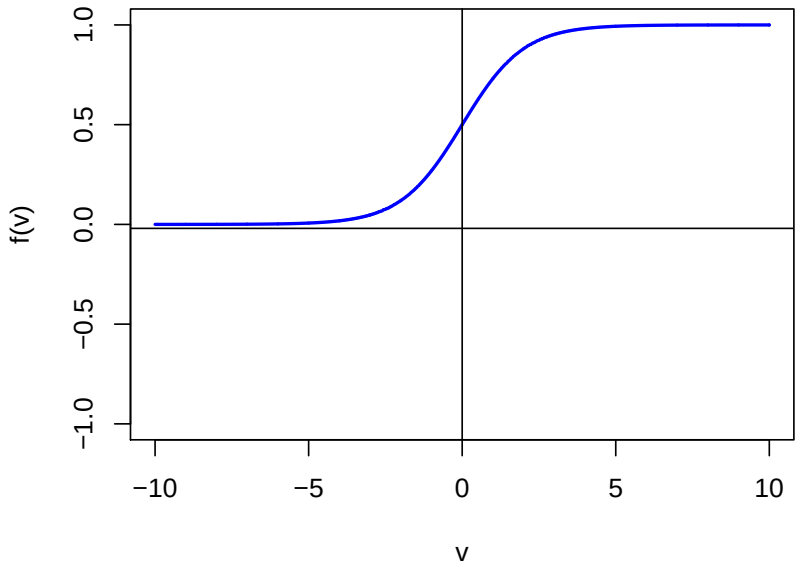


Figure: The sigmoid function

Activation function cont.

Hyperbolic Tangent Function or **tanh**: This is a very similar function to the previous sigmoid function and has much of the same properties. This function enables activation functions to range from -1 to $+1$.

$$f(v) = tanh(v)$$

Why is it used?

It also gives us a larger range of output to play with in the positive end of the scale meaning finer adjustments can be made. This function has a natural threshold of 0, meaning that any input which maps to a value greater than 0 is considered high (or 1) in binary terms.

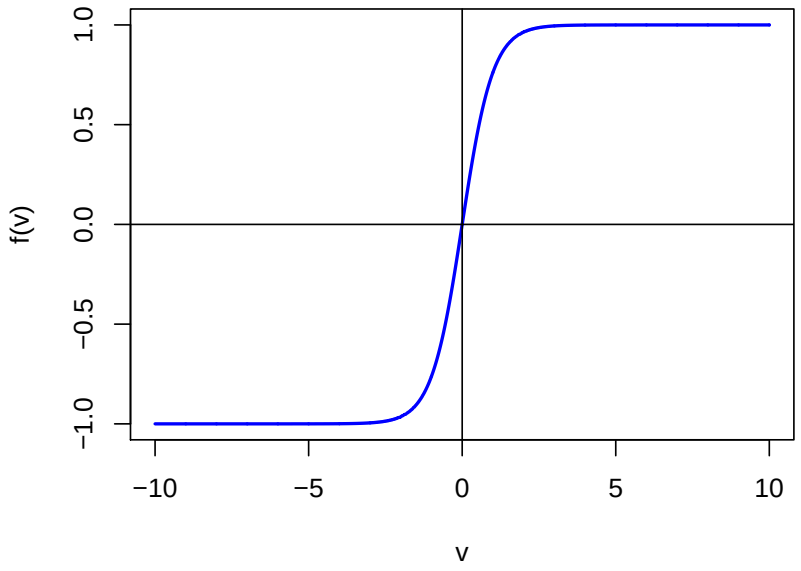


Figure: The tanh function

Activation function cont.

Whats the difference between the sigmoid and hyperbolic tangent? They both achieve a similar mapping, are both continuous, smooth, monotonic and differentiable, but give out different values. For a sigmoid function, a large negative input generates an almost zero output. This lack of output will affect all subsequent weights in the network which may not be desirable - effectively stopping the next nodes from learning. In contrast, the tanh function supplies -1 for negative values, maintaining the output of the node and allowing subsequent nodes to learn from it.

Activation function cont.

Identity/Linear function is a function that maps input to the same output value. It is a linear operator in vector space. Also, known straight line function where activation is proportional to the input.

$$f(v) = v$$

Why is it used?

If theres a situation where we want a node to give its output without applying any thresholds, then the identity (or linear) function is the way to go.

It is used in the final output layer nodes as we only want these nodes to do the input×weight operations before giving us its answer without any further modifications.

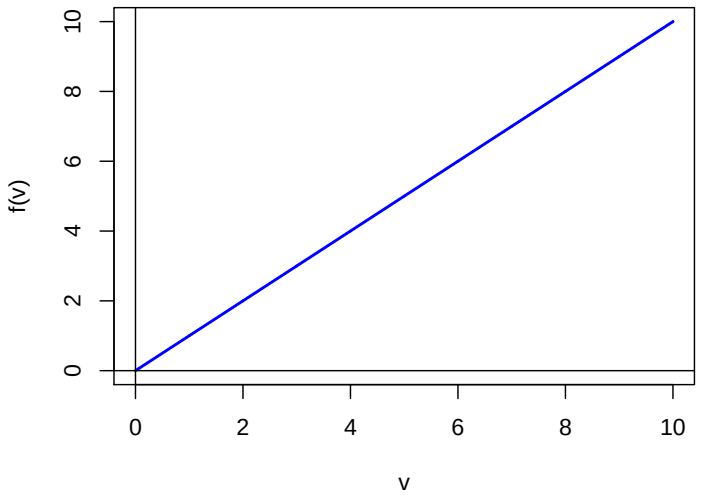


Figure: The identity function

Activation function cont.

Gaussian Function

$$f(v) = \exp\{-v^2\}$$

Why is it used?

The gaussian function is an even function, thus is gives the same output for equally positive and negative values of input. It gives its maximal output when there is no input and has decreasing output with increasing distance from zero. We can perhaps imagine this function is used in a node where the input feature is less likely to contribute to the final result.

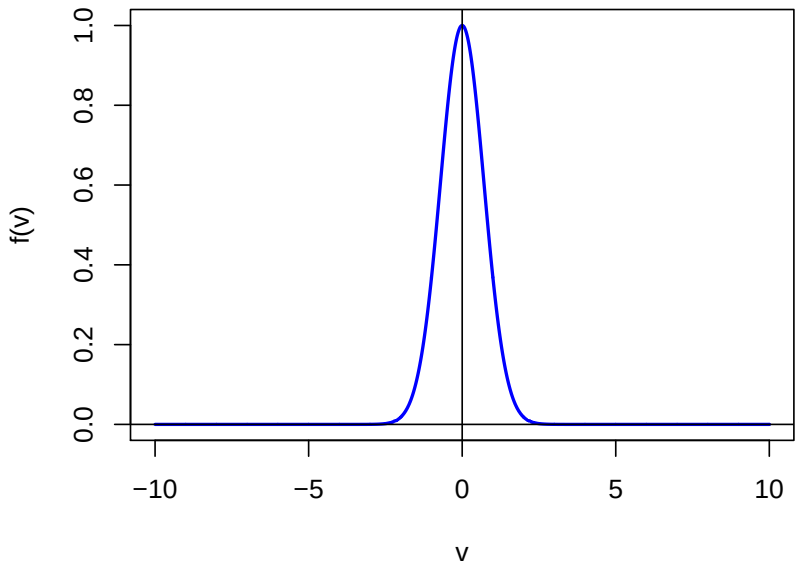


Figure: The Guassian function

Activation function cont.

Ramp Function:
The name of the ramp function is derived from the appearance of its graph. It maps negative inputs to 0 and positive inputs to the same output.

$$f(v) = \begin{cases} 0 & x < T_1 \\ \frac{v - T_1}{T_2 - T_1} & T_1 \leq v \leq T_2 \\ 1 & v > T_2 \end{cases}$$

Why is it used?

The ramp function is a truncated version of the linear function. From its shape, it looks like a more definitive version of the sigmoid function in that its maps a range of inputs to outputs over the range $(0, 1)$ but this time with definitive cut off points T_1 and T_2. It may not be common to see T_1 in the negative region unless the ramp is equally distributed about 0.

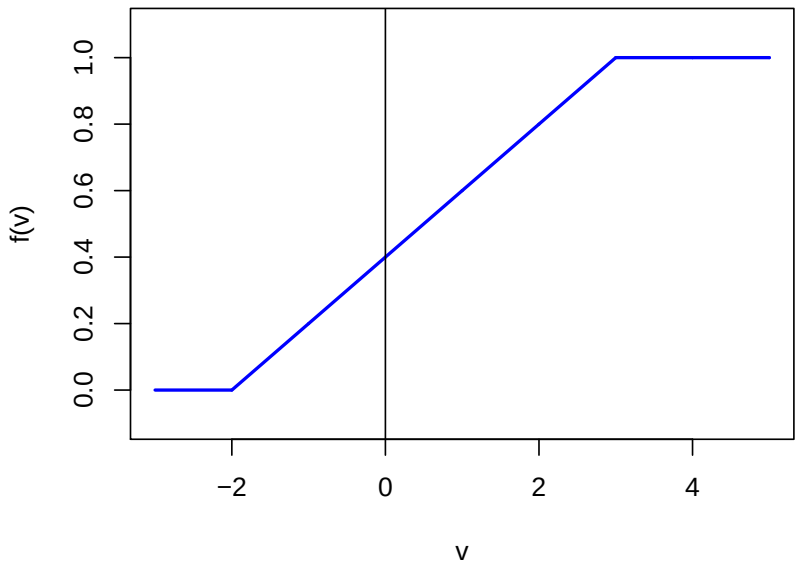


Figure: The ramp function for $T_1 = -2$ and $T_2 = 3$

Activation function cont.

ReLu function stands for the **R**ectified **L**inear **U**nit (ReLU). It is the most used activation function in the world. It output 0 for negative values of x.

In a ReLU, T1=0 and T2 is the maximum of the input giving a linear function with no negative values as

$$f(v) = \max(0, v)$$

Why is it used?
ReLUs are the smooth approximation to the the sum of many logistic units and produce sparse activity vectors.

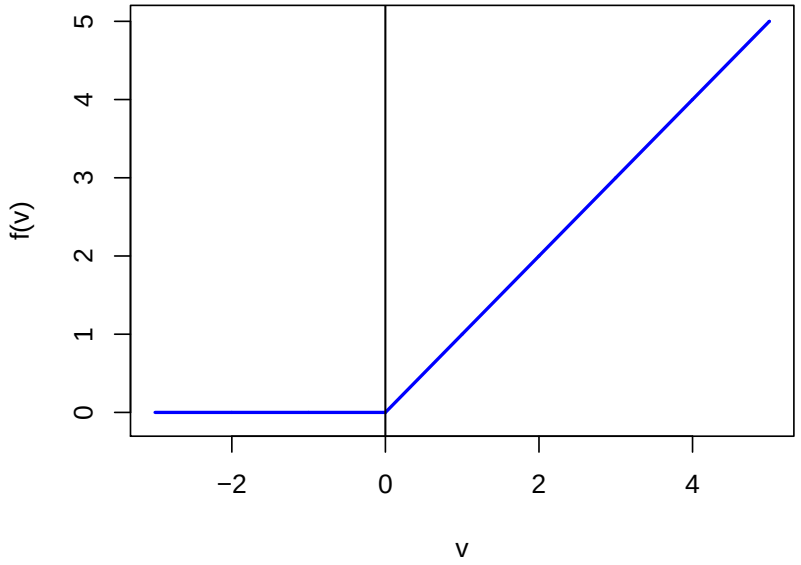


Figure: The relu function

Activation function cont.

Softplus or SmoothReLU function: It is a approximation to the relu function.

$$f(v) = \log(1 + \exp\{x\})$$

Why is it used?
Outputs produced by sigmoid and tanh functions have upper and lower limits whereas softplus function produces outputs in scale of $(0, +\infty)$. Thats the essental difference.

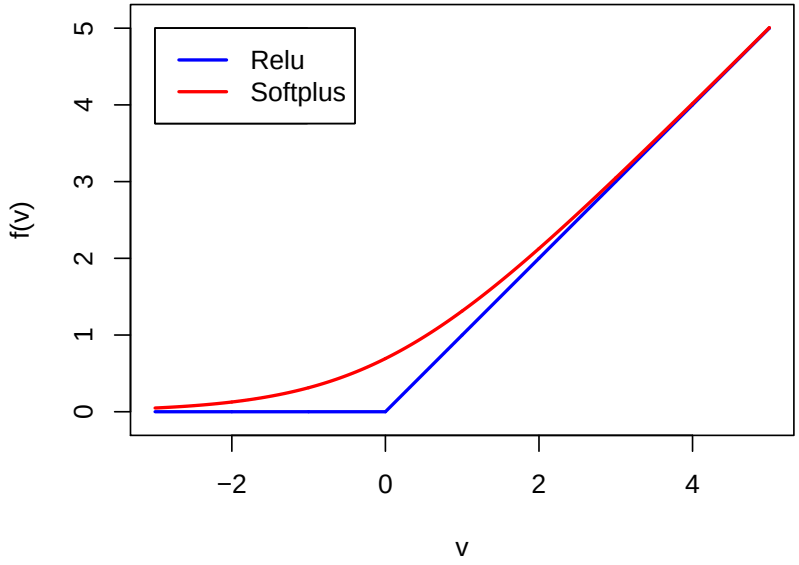


Figure: The softplus and relu functions

Multilayer perceptron (MLP)

Moving onwards, multi-layer perceptron, also known as feed-forward neural networks, consists of a sequence of layers each fully connected to the next one.

A multilayer perceptron (MLP) has one or more hidden layers along with the input and output layers, each layer contains several neurons that interconnect with each other by weight links.

The number of neurons in the input layer will be the number of attributes in the dataset, neurons in the output layer will be the number of classes given in the dataset.

MLP cont.

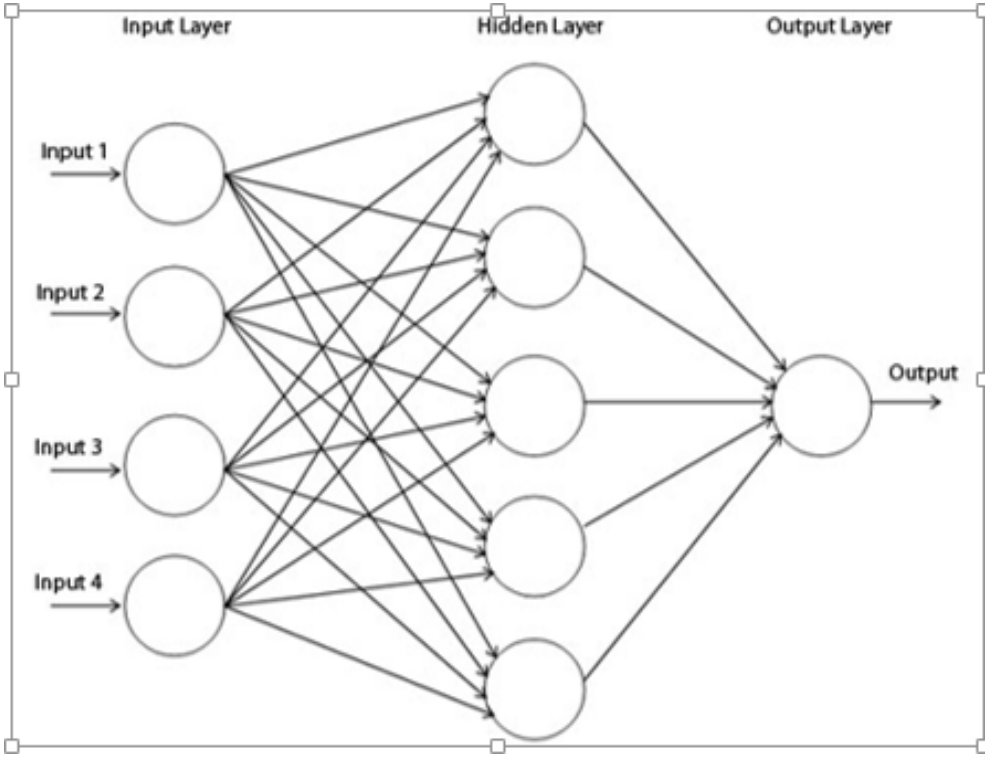
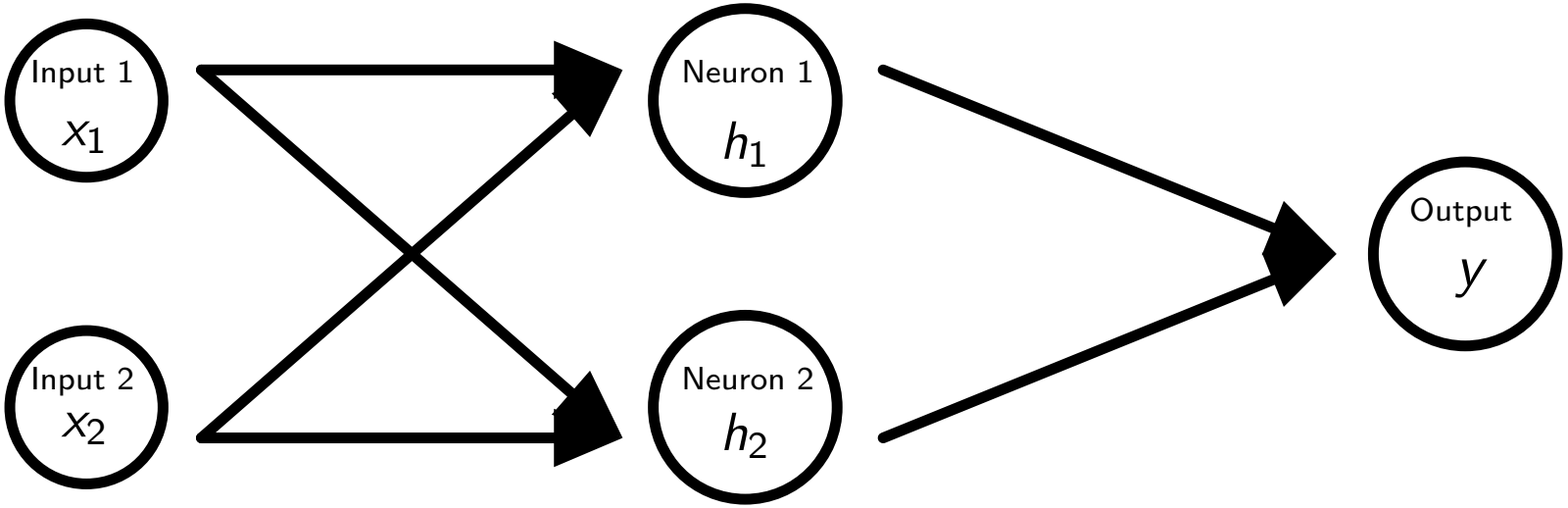
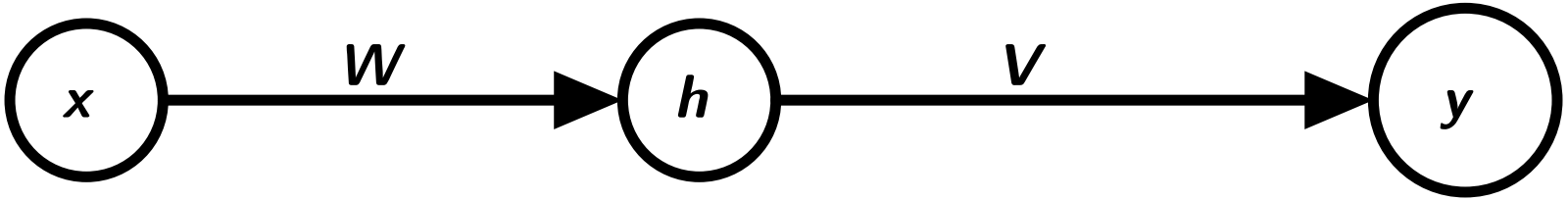


Figure: Multilayer Perceptron

Figure shows a multilayer perceptron where we have three layers at least and each layer is connected to the last one. To make the architecture deep, we need to introduce multiple hidden layers.

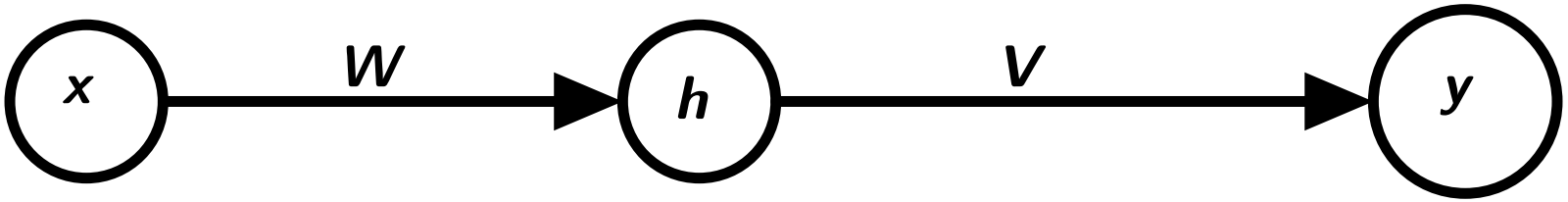
Multi-layer perceptron (MLP):





XOR

The aim of the training process is to find the optimum network parameters so that with the given $\boldsymbol{x}$, the network generates the following $\boldsymbol{y}$.



$$\boldsymbol{X} = \begin{bmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{bmatrix} \quad \boldsymbol{y} = \begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}$$

What is the equation for in terms of network parameters?

XOR

$$\begin{bmatrix} h_1 \\ h_2 \end{bmatrix} = f_1\left(\begin{bmatrix} W_{11} & W_{21} \\ W_{12} & W_{22} \end{bmatrix}\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} c_1 \\ c_2 \end{bmatrix}\right) \quad y = f_2\left(\begin{bmatrix} v_1 & v_2 \end{bmatrix}\begin{bmatrix} h_1 \\ h_2 \end{bmatrix} + \begin{bmatrix} b_1 \\ b_2 \end{bmatrix}\right)$$

$$\boldsymbol{h} = f_1(\boldsymbol{W}^\top \boldsymbol{x} + \boldsymbol{c}) \quad \boldsymbol{y} = f_2(\boldsymbol{V}^\top \boldsymbol{h} + \boldsymbol{b})$$

If f_1 is Relu and f_2 is the linear function, then

$$\boldsymbol{y} = f(\boldsymbol{x}, \boldsymbol{W}, \boldsymbol{c}, \boldsymbol{V}, \boldsymbol{b}) = \boldsymbol{V}^\top \max(0, \boldsymbol{W}^\top \boldsymbol{x} + \boldsymbol{c}) + \boldsymbol{b}$$

$$\boldsymbol{W} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \quad \boldsymbol{c} = \begin{bmatrix} 0 \\ -1 \end{bmatrix}, \quad \boldsymbol{V} = \begin{bmatrix} 1 & -2 \end{bmatrix}$$

recognize digits by neural network

https://www.cs.ryerson.ca/~aharley/vis/fc/

Number of Neurons

Many neurons:
- Higher accuracy
- Slower
- Risk of over-fitting
  - Memorizing rather than understanding
  - The network will be useless with new problem

Few neurons:
- Lower accuracy
- Inability to learn at all

Optimal number

Effect of number of layers, neurons

https://playground.tensorflow.org/

Loss functions for NN

Regression:
- Use the same Loss as Linear Regression
- Quadratic loss (i.e. mean squared error)

Classification:
- Use the same Loss as Logistic Regression
- Cross-entropy (i.e. negative log likelihood)

Quadratic	$L(y, \hat{y}) = (y - \hat{y})^2$	$\frac{dL}{dy} = y - \hat{y}$
Cross-Entropy	$L(y, \hat{y}) = \hat{y} \log(y) + (1 - \hat{y}) \log(1 - y)$	$\frac{dL}{dy} = \hat{y} \frac{1}{y} + (1 - \hat{y}) \frac{1}{y-1}$

Error Estimation

The root mean square error (RMSE) is a frequently-used measure of the differences between values predicted by a model or an estimator and the values actually observed from the thing being modeled or estimated.

Weights Adjusting

After each iteration, weights should be adjusted to minimize the error: (1) All possible weights, (2) Back propagation

Back-propagation is used at each layer to minimize the error between the layers response and the actual data.

The error at each hidden layer is an average of the evaluated error.

Hidden layer networks are trained this way.

Back Propagation

- N is a neuron.
- N_w is one of N inputs weights
- N_{out} is N's output.
- $N_w = N_w + \nabla N_w$
- $\nabla N_w = N_{\text{out}} * (1 - N_{\text{out}}) * N_{\text{ErrorFactor}}$
- $N_{\text{ErrorFactor}} = N_{\text{ExpectedOutput}} - N_{\text{ActualOutput}}$
- This works only for the last layer, as we can know the actual output, and the expected output.

A recipe for NN

1. Given training data: $\{(\boldsymbol{x}_i, y_i); i = 1, \ldots, N\}$
2. Choose each of these:
   - Decision function: $\hat{y} = f(\boldsymbol{x}_i, \boldsymbol{w}, b)$
   - Loss function: $L(y_i, \hat{y}_i) \in \mathbb{R}$
3. Define goal: $(\boldsymbol{w}, b) = \text{argmin}_{\boldsymbol{w},b} \sum_{i=1}^{N} L(f(\boldsymbol{x}_i, \boldsymbol{w}, b), y_i)$
4. Train with SGD (take small steps opposite the gradient)

$$\boldsymbol{w}^{(t+1)} = \boldsymbol{w}^{(t)} - \eta_t \nabla(f(\boldsymbol{x}_i, \boldsymbol{w}, b), y_i)$$

Implementation of a NN in R

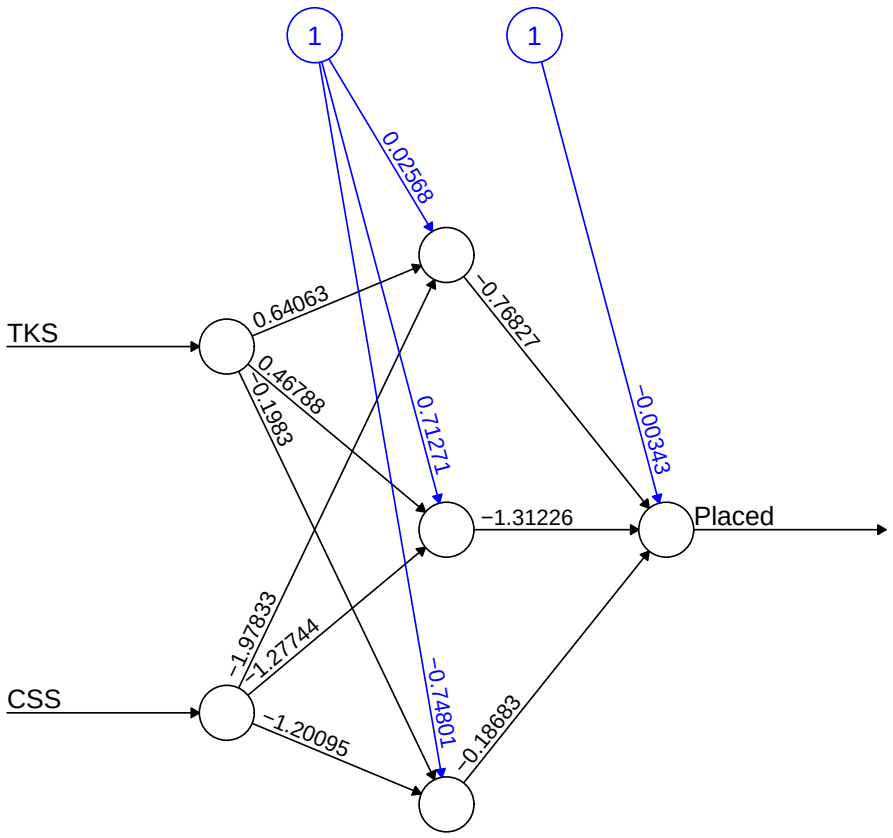
Create training dataset (Lab16)
Let us create the dataset. Here you need two kinds of attributes or columns in your data: Feature and label. In the table shown above, you can see the technical knowledge, communication skills score and placement status of the student. So the first two columns (Technical Knowledge Score and Communication Skills Score) are features and third column (Student Placed) is the binary label.

Implementation cont.

Technical Knowlege Score	Communication Skills Score	Student Placed
20	90	Placed
10	20	Not Placed
30	40	Not Placed
20	50	Not Placed
80	50	Placed
30	80	Placed

Implementation cont.

Building a NN classifier model:
First, import the neuralnet library and create NN classifier model by passing argument set of label and features, dataset, number of neurons in hidden layers, and error calculation.



Implementation cont.

Create test dataset

Create test dataset using two features Technical Knowledge Score and Communication Skills Score.

Predict the results for the test set

Predict the probability score for the test data using the compute function.
Now, Convert probabilities into binary classes.
Predicted results are 1,0, and 1.

Use-cases of NN

NN's wonderful properties offer many applications such as:

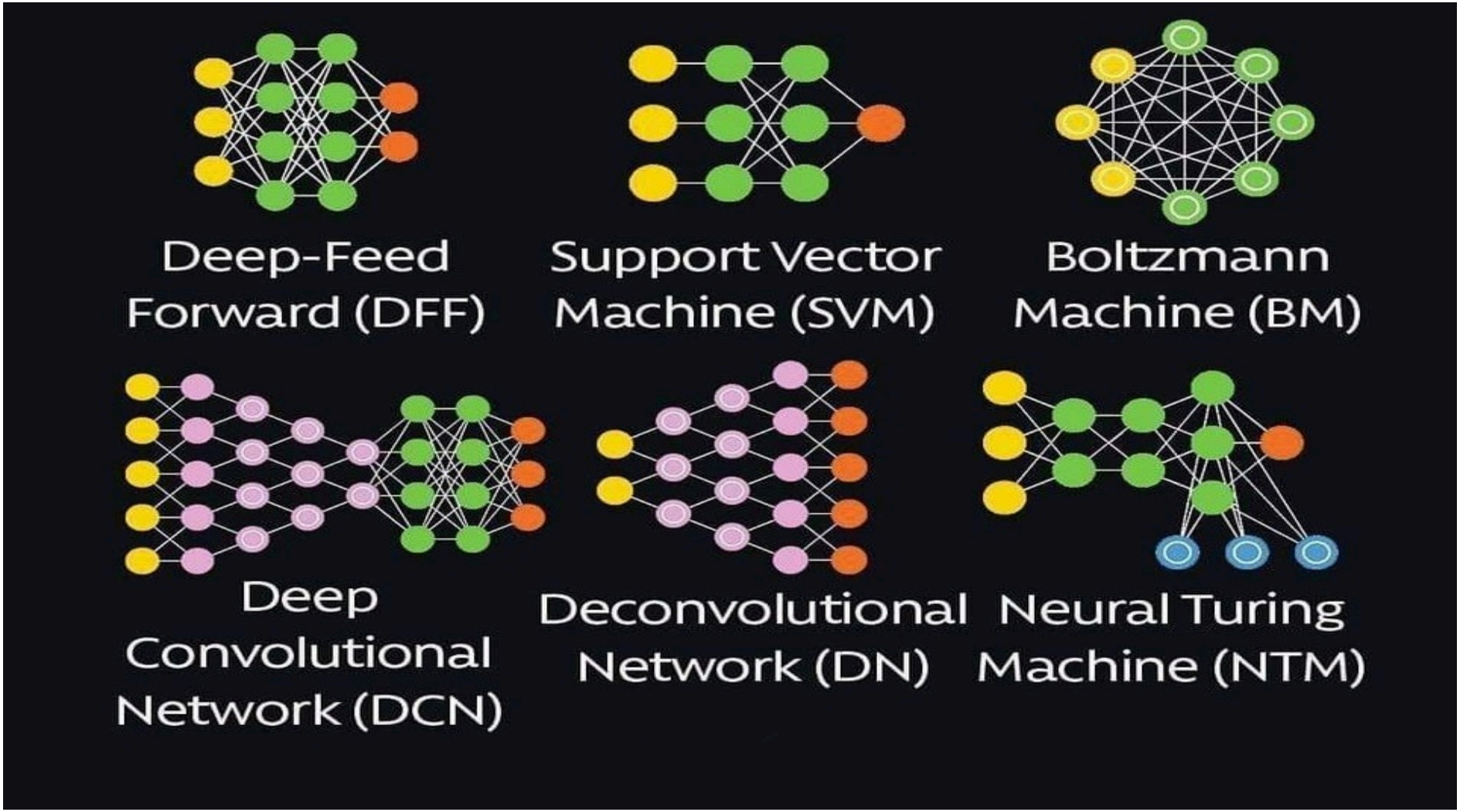
Pattern Recognition: Neural networks are very suitable for pattern recognition problems such as facial recognition, object detection, fingerprint recognition, etc.

Anomaly Detection: Neural networks are good at pattern detection, and they can easily detect the unusual patterns that dont fit in the general patterns.

Time Series Prediction: Neural networks can be used to predict time series problems such as stock price, weather forecasting.

Natural Language Processing: Neural networks offer a wide range of applications in Natural Language Processing tasks such as text classification, Named Entity Recognition (NER), Part-of-Speech Tagging, Speech Recognition, and Spell Checking.

Types of Neural networks



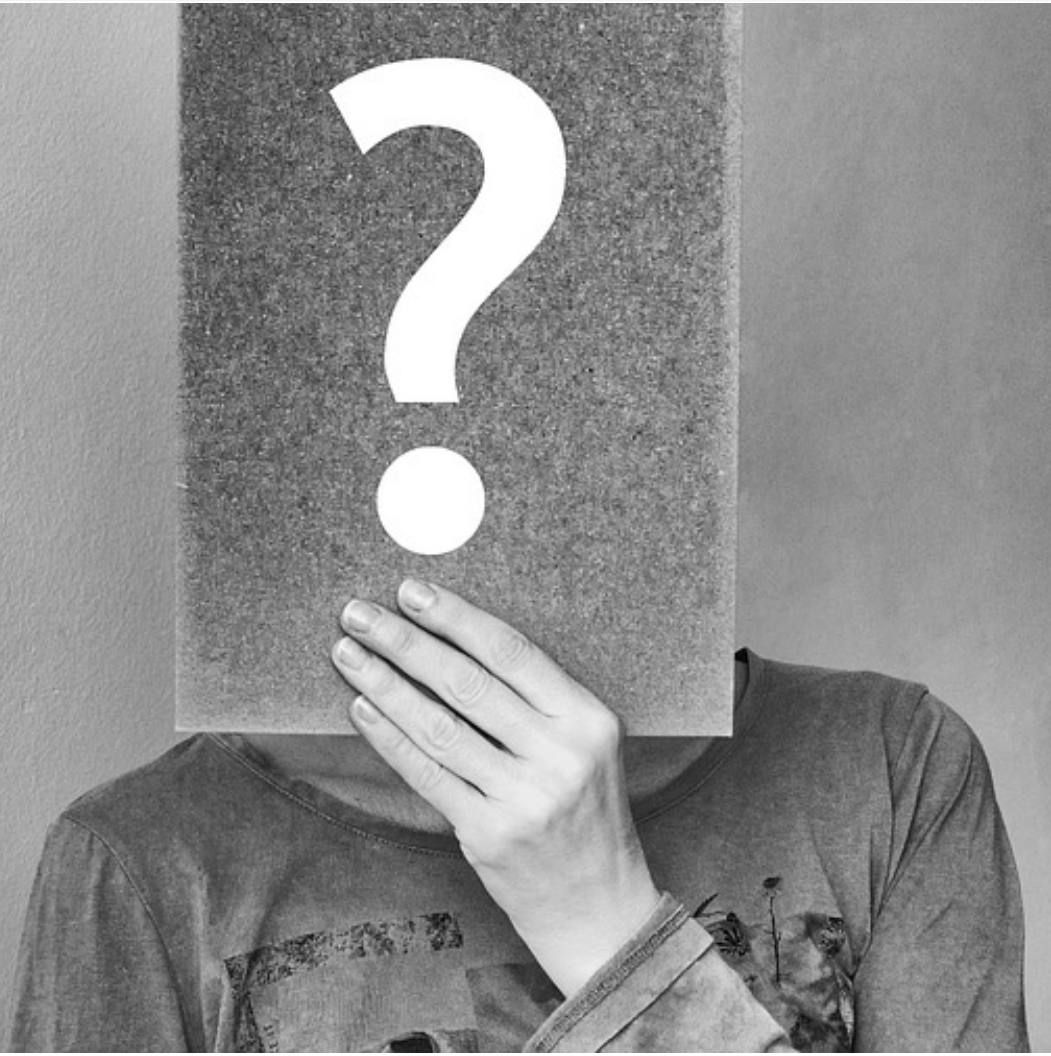
See https://www.asimovinstitute.org/neural-network-zoo/

Pros and Cons

Neural networks are more flexible and can be used with both regression and classification problems. NNs are good for the nonlinear dataset with a large number of inputs such as images. NNs can work with any number of inputs and layers. NNs have the numerical strength that can perform jobs in parallel.

There are more alternative algorithms such as SVM, Decision Tree, and Regression are available that are simple, fast, easy to train, and provide better performance. NN are much more of the black box, require more time for development and more computation power. NNs requires more data than other Machine Learning algorithms. NNs can be used only with numerical inputs and non-missing value datasets. A well-known NN researcher said "A neural network is the second best way to solve any problem. The best way is to actually understand the problem".

Main reference

I used
https://www.datacamp.com/community/tutorials/neural-network-m

Thank you for your attention.